

Preguntas y respuestas para el comité

Hito 3 — MVP Multinube RutaExpress

Apoyo oral. Si te trabas, lee solo la línea **Qué decir**.

Regla de oro: distingue siempre *diseño* vs *lo que la demo muestra hoy*. Si algo es parcial, dilo. El comité castiga más inventar que admitir alcance.

Evitar	Mejor
«Porque lo dice el Hito 2 / Alternativa A»	«Porque el cliente ya tiene ese dolor / esa nube»
«Porque lo documentamos»	«Si elegimos lo otro, se rompe X escenario o cuesta Y»

0. Preguntas generales (las que más hace el comité)

El comité suele preguntar **abierto**, no “¿por qué AKS y no Functions?”. Responde en 3–4 frases: **qué / por qué / qué falta**.

¿Qué problema resuelven?

Qué decir: RutaExpress pierde plata y confianza por pedidos duplicados, picos tipo Cyber Days, firmas que no llegan cuando no hay red, y un tracking que no coincide entre sistemas. El MVP ataca ese núcleo: orden bien creada, stock reservado, eventos confiables, entrega offline y seguimiento.

Expliquen la arquitectura en un minuto

Qué decir: Azure es el hub de pedidos (APIM → AKS → SQL → Event Hubs). AWS es la última milla del conductor (app, evidencias en S3). GCP prepara la lectura de tracking (BigQuery). No buscamos tres nubes “por moda”: el cliente ya está ahí; nosotros unimos con eventos en vez de integraciones punto a punto.

¿Qué seguridad tienen?

Qué decir: Entrada por gateway (APIM), tráfico cifrado (HTTPS), secretos fuera del código (Key Vault / Secrets Manager), roles con mínimo privilegio, y colas con dead-letter para no perder mensajes. Identidad completa tipo login empresarial (Entra ID / OAuth) está en el diseño, no la vendemos como 100 % cerrada en la demo.

¿Quién es el responsable de seguridad?

Qué decir: En el modelo TO BE, la seguridad transversal es de **plataforma**: iniciativa **INI-05** / capacidad **PLT-02** (identidad y accesos: Entra ID + Key Vault, políticas en APIM). No es “el desarrollador del OMS solo”. Cada equipo de dominio aplica lo que plataforma define (secretos, TLS, roles); plataforma gobierna el estándar multinube. En el MVP académico lo opera el mismo equipo del proyecto, con esa separación lógica de responsabilidades.

¿Es difícil trasladar / migrar a otra nube?

Qué decir: Depende de **qué** nuevas. Lo difícil es el PaaS nativo (Event Hubs, SQS, BigQuery, AKS): eso sí amarra. Lo que facilita el cambio son los **contratos**: APIs, eventos JSON y adaptadores entre nubes. No prometemos “un clic y cambias de cloud”; sí que el diseño por dominios + eventos evita un monolito imposible de mover. Migrar un dominio (ej. solo última milla) es más realista que levantar todo de una.

¿Cómo despliegan? ¿Qué CI/CD tienen?

Qué decir: Infraestructura con Terraform. Al subir cambios: el pipeline formatea, valida y hace un plan (muestra qué cambiaría) **sin aplicar solo**. El apply es con revisión humana, primero Azure y después AWS/GCP. Después suben los servicios y se hacen pruebas básicas del flujo.

¿Qué estrategias de despliegue usan?

Qué decir: Tres capas:

1. **Infra (IaC):** Terraform — plan → aprobación → apply **secuencial** Azure → AWS → GCP (el puente necesita outputs del hub).
2. **Aplicaciones:** en Azure con **Helm/containers en AKS** (OMS, Inventario, bus-workers); en AWS el task **ECS Fargate**; en GCP **Cloud Run**. Imágenes a registry, no “copiar a mano”.
3. **Verificación:** smoke / escenarios E1–E8 después del deploy.

Estrategia de riesgo: no apply ciego; un ambiente **mvp**; cambios por PR. En producción el siguiente paso sería rolling/blue-green; en el MVP no prometemos canary completo — priorizamos orden, control y smoke.

¿Qué buenas prácticas usan al subir la infraestructura (IaC)?

Qué decir: Nada de secretos en el repositorio. Cambios por Pull Request. Primero se ve el plan, después se aplica. Apply ordenado por nube. Ambiente de demo etiquetado para controlar costos. No es “subí el código y ya está en la nube”.

¿Cómo probaron que funciona?

Qué decir: Con escenarios de aceptación E1–E8 (pedido OK, duplicados, fallo y dead-letter, offline, etc.). En la defensa mostramos el flujo principal y al menos un fallo controlado. El IaC se prueba con validate/plan en CI; el apply no es automático.

¿Qué está implementado y qué es solo diseño?

Qué decir: Implementado con más fuerza: flujo de órdenes en Azure, mocks, outbox hacia Event Hubs, demo de dead-letter. Parcial / objetivo: puente completo AWS→Azure y tracking real en BigQuery. Pedimos validar la arquitectura y la ruta, no un TO BE terminado.

¿Cuál es el valor / el aporte de este MVP?

Qué decir: Baja el riesgo de la transformación: demostramos el camino crítico multinube antes de migrar 26 apps. Si el patrón de eventos, Saga y offline no funciona aquí, no tiene sentido invertir el plan de 36 meses a ciegas.

¿Cómo escalan si crece el volumen?

Qué decir: Escalamos por capa: más nodos/pods en AKS, más throughput en Event Hubs, colas SQS para el móvil, BigQuery para lecturas masivas. El SKU de la demo es chico a propósito; el diseño permite subir capacidad sin redibujar todo.

¿Cómo aseguran calidad / no romper producción?

Qué decir: En este hito es ambiente mvp, no producción bancaria. Aun así: cambios por PR, plan de Terraform, apply con aprobación, smoke del flujo, y separación diseño vs lo cableado. En un siguiente paso entrarían ambientes dev/qa/prod y más pruebas automatizadas.

¿Alta disponibilidad? ¿Qué pasa si algo cae?

Qué decir: En demo no prometemos multi-región. Sí hay resiliencia de mensajería (reintentos, DLQ, offline en el teléfono). Si cae el hub Azure, el conductor aún puede registrar en el móvil y sincronizar a AWS; se degrada el tracking canónico. HA fuerte es post-MVP.

¿Cómo controlan costos?

Qué decir: SKUs pequeños, una sola región de demo por cercanía de nubes, tags de ambiente, presupuesto ~449 USD/mes de nube, y no aplicar infra sin ver el plan.

¿Cumplen el enunciado del curso?

Qué decir: Sí en lo pedido para el hito: arquitectura multinube, laC, MVP que demuestra el núcleo, C4, costos y argumentación. Los mocks están permitidos. No afirmamos migración completa del AS IS.

¿Qué riesgos ven y cómo los mitigan?

Qué decir: Tres grandes: (1) integración entre nubes más frágil que un solo vendor → eventos + adaptadores; (2) picos de campaña → colas y DLQ; (3) última milla sin red → offline y store-and-forward. El riesgo de “todo dibujado y nada corre” lo bajamos con demo e laC reales.

¿Qué harían distinto o qué sigue después?

Qué decir: Cerrar identidad end-to-end, cablear del todo el puente AWS→Azure, proyección real a BigQuery, y subir SKUs/HA cuando el patrón esté validado. También WMS real en lugar del mock.

¿Cómo trabajaron / cómo se organizaron?

Qué decir: Por frentes alineados a la arquitectura: hub Azure, última milla AWS, lectura GCP, laC/costos y demo/comité. La fuente de verdad son los docs del hito + el repo de implementación; la demo es la evidencia oral.

¿Dónde está la evidencia (no solo PowerPoint)?

Qué decir: Carpeta **Implementacion** (Terraform y servicios), workflow de GitHub Actions, demo web (local y/o AKS), y el paquete Hito 3 (dossier, C4, costos, esta guía).

¿Por qué deberíamos aprobarles el hito?

Qué decir: Porque el problema del cliente está bien diagnosticado, la solución multinube es coherente con el AS IS, el MVP prueba el flujo que importa, hay IaC y disciplina de despliegue, y somos honestos con lo parcial. No vendemos humo de “ya está en producción”.

1. Alcance del MVP

¿Esto es el sistema final de 36 meses?

Qué decir: No. El TO BE es un plan de años. El MVP es una maqueta que prueba el núcleo: pedido → reserva de stock → evento → entrega con offline → tracking. No migramos las 26 aplicaciones.

¿Por qué usan mocks (WMS, ERP, portal, TMS)?

Qué decir: El enunciado lo permite. Los sistemas reales están on-premises y no están en alcance. Con mocks probamos contratos HTTP y eventos (Saga, circuit breaker, tracking) sin VPN ni migración del centro de datos.

¿Qué son E1–E8?

Qué decir: Son los criterios de aceptación de la demo. Cada uno ataca un dolor del caso: pedidos duplicados, picos tipo Cyber Days, firmas perdidas, tracking inconsistente.

¿Qué queda fuera del MVP?

Qué decir: WMS cloud real, optimizador de rutas con ML, liquidación automática, observabilidad completa y Pub/Sub en GCP en la v1.

2. Multinube

¿Por qué tres nubes y no todo en una?

Qué decir: El cliente **ya** opera en Azure, AWS y GCP. El problema no es “tener tres proveedores”, sino que hoy se integran mal (punto a punto). Azure concentra órdenes y el bus; AWS la última milla y evidencias; GCP la lectura analítica. Unirlas con eventos es más realista que migrar todo antes de probar.

Frase de 20 s: «Falló la integración, no el hecho de ser multinube. El MVP demuestra el remedio: hub por eventos + cada carga donde ya vive.»

¿No es más caro y complejo?

Qué decir: Sí hay costo de tráfico entre nubes, pero en la demo es bajo (~18 USD/mes). Es el precio de **no** migrar de golpe 68 mil entregas/día y el S3. Mitigamos con eventos chicos y regiones cercanas (eastus / us-east-1).

¿Azure no es un SPOF (punto único de falla)?

Qué decir: El hub de negocio (orden, bus) está donde ya está el OMS, en Azure. Mitigamos con colas, DLQ (mensajes fallidos) y reproceso. Multi-región es post-MVP.

¿Por qué el conductor no habla directo con Azure?

Qué decir: En última milla necesita confirmar rápido al celular y guardar fotos cerca de donde ya está la app (AWS/S3). Si cada entrega dependiera de Azure, se cae con mala red — el caso ya perdió miles de firmas por eso. Flujo: móvil → AWS → luego evento al hub.

¿La integración entre nubes ya está cableada de punta a punta?

Qué decir: El diseño sí: móvil → SQS → retry-worker → EventBridge → adaptador → Event Hubs, y de Event Hubs hacia BigQuery. En el MVP eso está **parcial / objetivo**. Lo que sí demostramos fuerte es el flujo de órdenes en Azure y la base en AWS/GCP.

3. Azure — ¿por qué AKS?

¿Por qué AKS y no Azure Functions o App Service?

Qué decir: En Azure corren varios servicios a la vez: OMS, Inventario y workers del bus. Necesitamos procesos que viven mucho rato (Saga esperando al WMS, outbox publicando eventos), no una función que arranca y muere en segundos. AKS nos da un solo cluster pequeño para eso. Functions encaja para un endpoint puntual; aquí el dominio es de vida larga.

¿Por qué Event Hubs y también Service Bus?

Qué decir: No son lo mismo. Event Hubs = río de eventos de alto volumen (stream). Service Bus = cola con acuse y dead-letter (mensaje fallido). Cada uno cumple un rol.

¿Por qué tanta lógica en el bus (DLQ, replay)?

Qué decir: En Cyber Days el caso necesita no perder mensajes y poder reprocesar con control. Una cola sola no cuenta la historia de E5 (fallo → dead-letter → reproceso).

¿OMS e Inventario comparten la misma SQL?

Qué decir: Sí, por costo del MVP: una instancia, esquemas separados, sin joins entre dominios. Se hablan por HTTP/eventos. Separar bases después no cambia el diseño de dominio.

¿Para qué Redis si ya hay SQL?

Qué decir: Para no duplicar pedidos en milisegundos (idempotencia / dedup) sin golpear SQL en cada reintento del cliente.

¿Por qué APIM en SKU Developer?

Qué decir: Es prototipo académico, sin SLA de producción. Cuesta ~50 USD/mes vs ~700 del Standard. Alcanza para gateway, mocks y demo.

¿Por qué la Saga llama al WMS por HTTP síncrono?

Qué decir: En el negocio el almacén confirma en el momento. Si fuera solo async, no sabríamos al tiro si la orden quedó confirmada y la compensación se complica. En la demo el mock responde OK o 503/timeout (E4).

4. AWS — última milla

¿Por qué ECS Fargate y no Lambda?

Qué decir: El móvil necesita una API que responda al tiro (ACK), suba evidencias a S3 y, en el mismo servicio, reintente el puente a Azure. Lambda trocea eso en muchas funciones y complica el offline. Fargate mantiene un contenedor siempre listo para la demo.

¿Por qué no EKS (Kubernetes en AWS) si ya tienen AKS?

Qué decir: Kubernetes ya está donde hace falta: AKS con OMS, inventario y bus. En AWS solo hay **una** app (móvil). EKS sería un segundo cluster caro (~73 USD/mes solo el control plane) sin beneficio. Mismo criterio en GCP: Cloud Run, no GKE.

¿DynamoDB o una base SQL en AWS?

Qué decir: DynamoDB para outbox y ACK: acceso por clave, barato on-demand, sin administrar otro SQL solo para colas pendientes.

¿El offline es DynamoDB?

Qué decir: No. Offline = el teléfono (SQLite). DynamoDB entra cuando hay red hacia AWS.

¿Para qué SQS y EventBridge?

Qué decir: SQS aguanta picos del móvil. EventBridge publica hacia el hub. Si el puente a Azure falla, no se cae la app del conductor.

5. GCP

¿Qué hace GCP en el MVP?

Qué decir: Prepara la lectura analítica (CQRS): Cloud Run + BigQuery. No procesa órdenes. El tracking “de verdad” sobre BigQuery sigue siendo objetivo; hoy el portal de seguimiento del cliente es un mock.

¿Por qué Cloud Run y no GKE?

Qué decir: Es un handler liviano que puede escalar a cero. GKE sería exceso para un proyector mínimo.

¿Por qué no leer tracking directo de SQL Azure?

Qué decir: Miles de consultas de tracking no deben frenar la base transaccional del OMS. CQRS separa: escribir en SQL, leer en BigQuery.

6. Patrones

¿Por qué tantos patrones (Saga, CQRS, outbox...)?

Qué decir: Porque el caso ya está distribuido: orden, stock, bus, móvil, lectura. Cada patrón ataca un dolor concreto (duplicados, desync, offline, tracking inconsistente). No son adorno.

¿Por qué hay outbox en Azure y en AWS?

Qué decir: Mismo idea: no perder el evento si el broker falla después de guardar en base. Distinto almacén (SQL vs DynamoDB), mismo patrón.

¿Idempotencia y deduplicación no son lo mismo?

Qué decir: Casi, pero el caso tiene dos problemas: (E1) el cliente reintenta con la misma clave; (E2) crea "otra" orden distinta y genera 32 mil duplicados. Hay que cubrir ambos.

¿CQRS no duplica datos?

Qué decir: Sí, a propósito. La verdad del pedido vive en SQL; BigQuery es una copia para consultas. Puede ir un poco atrasada (eventual), y está bien.

7. Diagramas C4

¿Por qué en el Nivel 3 hay 4 diagramas?

Qué decir: Porque N3 no es "toda la plataforma otra vez": es un **zoom por contenedor crítico**. Son cuatro porque el MVP gira en torno a cuatro piezas distintas: **OMS** (crear pedido y Saga), **Inventario** (reservar/liberar stock), **bus-workers** (sacar el evento del outbox a Event Hubs) y **móvil** (entrega offline y evidencias). Si metiéramos todo en un solo N3, el diagrama sería ilegible y mezclaría dominios. N2 muestra el mapa; cada N3 explica un caso.

¿Por qué tantas cajas en N3?

Qué decir: Son módulos lógicos (responsabilidades dentro del código), no un pod por caja. En realidad desplegamos pocos contenedores.

¿Por qué en N3 del OMS no se ve Key Vault?

Qué decir: N3 hace zoom a un contenedor. La seguridad transversal se ve en N2 (PLT-02 / Key Vault) y en la config de APIM.

¿Query API es lo mismo que tracking E8?

Qué decir: No. Query API = soporte sobre SQL. E8 = lectura CQRS en BigQuery (objetivo). El mock-portal actual no consulta BigQuery.

8. Seguridad

¿Qué seguridad tienen implementada de verdad?

Qué decir:

1. Entrada por **APIM** (no dejamos APIs sueltas a internet).
2. **HTTPS / TLS** en tránsito.
3. **Secretos fuera del código** (Key Vault en Azure; Secrets Manager en AWS; Secret Manager en GCP).

4. Evidencias en **S3 cifrado con KMS** (diseño; la cadena completa aún es parcial).
5. Roles con **mínimo privilegio**.
6. **DLQ** para no perder mensajes fallidos.

Qué no digas como “ya listo”: login completo con Entra ID / OAuth/JWT para toda la demo, MFA, WAF, SIEM. Eso es diseño PLT-02; el MVP es parcial.

¿Ya usan OAuth / Entra ID?

Qué decir: Está en el diseño (APIM + identidad). En el diagrama N2, PLT-02 se ve principalmente como Key Vault. No afirmen que Entra ya autentica toda la demo si no lo muestran en vivo.

¿Las firmas/fotos están seguras?

Qué decir: El diseño es S3 con cifrado KMS y hash SHA-256. En la maqueta la cadena durable completa todavía es parcial.

9. Costos, laC y CI/CD

¿Cuánto cuesta?

Qué decir: Nube del MVP ~449 USD/mes. TCO año 1 (con gente, Lima) ~37 mil USD. ROI del orden de 164 %. Detalle en el doc de costos.

¿Todo está en infraestructura como código?

Qué decir: Sí es el requisito y tenemos módulos Terraform por nube en [Implementacion/terraform](#).

¿Cómo prueban / validan el laC?

Qué decir (simple): Cuando alguien sube cambios de Terraform:

1. El pipeline **formatea** el código (**fmt**) — que esté ordenado.
2. **Valida** la sintaxis (**validate**) — que Terraform entienda el archivo.
3. En el Pull Request hace un **plan** — muestra qué se crearía/cambiaría **sin aplicar**.
4. **No hacemos apply automático** al subir. Primero alguien revisa.
5. El diseño de despliegue es: aprobar → apply Azure → luego AWS → luego GCP → desplegar apps → pruebas smoke E1–E8.

Frase corta: «Subir laC no es desplegar. Primero se revisa el plan; el apply es decisión humana.»

¿Buenas prácticas de CI/CD al subir laC?

Qué decir:

1. Solo corre el pipeline si cambió la carpeta de Terraform.
2. Secretos en GitHub Secrets, nunca en el repo.
3. PR = fmt + validate + plan (sin apply).
4. Apply ordenado: primero Azure (hub), después AWS y GCP.
5. Después de la infra, se despliegan los servicios y se hacen smoke tests.

10. Preguntas incómodas (ensayarlas)

¿Por qué no todo en Azure y listo?

Qué decir: Porque el AS IS ya tiene móvil/S3 en AWS y analítica en GCP. Meter todo en Azure primero sería migrar años antes de probar el bus. El caso pide integración multinube, no un lift-and-shift completo.

¿Por qué no Kubernetes en las tres nubes?

Qué decir: Solo el hub transaccional justifica un cluster (AKS). Una sola app móvil no merece EKS. Un proyector liviano no merece GKE. Usamos el runtime mínimo por carga.

¿Microservicios no es overkill para un MVP?

Qué decir: El negocio ya tiene dominios separados (orden, inventario, móvil). No hicimos veinte servicios: son pocos workloads alineados a esos dominios.

¿Cuánto está implementado vs dibujado?

Qué decir: Lo más sólido es el flujo de órdenes en Azure (OMS, inventario HTTP, outbox → Event Hubs, mocks, demo E5 con DLQ). Puentes AWS→Azure y proyección BigQuery están en diseño / parciales. Pedimos validar decisiones y ruta evolutiva, no un TO BE 100 % cableado.

¿Si cae Azure, qué pasa con el conductor?

Qué decir: El conductor sigue pudiendo trabajar offline en el teléfono y sincronizar a AWS. Lo que se degrada es el hub de órdenes/tracking canónico. Por eso el ACK local y store-and-forward existen. Recuperación multi-región del hub es post-MVP.

¿Latencia entre nubes?

Qué decir: Aceptamos latencia en el puente async (eventos). Lo síncrono crítico (pedido, reserva, ACK al conductor) queda dentro de su nube. No hacemos chatty sync Azure↔AWS por cada clic.

¿Vendor lock-in?

Qué decir: Usamos PaaS de cada nube a propósito (Event Hubs, SQS, BigQuery). El antídoto no es abstraer todo: es contratos de evento claros y poder cambiar un adaptador. El MVP prioriza demostrar el flujo, no un anti-lock-in teórico.

¿Quién opera cada nube?

Qué decir: En el modelo TO BE, Azure concentra plataforma/OMS; AWS última milla; GCP analítica. En el MVP académico el mismo equipo opera las tres con IaC y roles mínimos.

¿Cómo demuestran E1–E8 en 5 minutos?

Qué decir: Priorizamos el happy path de orden + un fallo controlado (DLQ/E5) y, si hay tiempo, offline/evidencia. No pretendemos correr los ocho en vivo; están mapeados en el dossier y la demo los cubre por escenarios.

¿Rollback si Terraform sale mal?

Qué decir: Plan antes de apply, apply por nube (no las tres a ciegas), estado remoto, y cambios por PR. Si algo falla, se corrige el plan o se destruye el recurso puntual del ambiente mvp — no es producción bancaria.

¿Observabilidad?

Qué decir: Diseño con Monitor / App Insights / logs. En el MVP no afirmamos observabilidad plena (está fuera o parcial). Para la defensa: health de servicios y evidencias de demo bastan; SIEM completo es INI posterior.

¿Datos personales / Perú?

Qué decir: Ambiente mvp de demo en US-East por precio y cercanía entre nubes. En producción se definirían región y retención según política del cliente; no es el foco de esta maqueta.

¿Por qué Event Hubs y no Kafka propio?

Qué decir: Menos operación, encaja con Azure, suficiente para el volumen de la demo. Kafka self-managed sería más costo y complejidad sin beneficio en el MVP.

¿El comité ve código o solo diagramas?

Qué decir: Hay laC, servicios en **Implementación/**, demo web y despliegue en AKS. Los diagramas C4 explican; el código y la demo evidencian el núcleo.

11. Más preguntas que sí te pueden hacer

¿Qué es exactamente la Saga aquí?

Qué decir: Es la orquestación del pedido: reservar stock y confirmar almacén. Si el WMS falla después de reservar, se hace Release (compensación) y se libera el stock. No es “un product name”: es el patrón de pasos con deshacer.

¿Qué es el outbox?

Qué decir: Guardamos el evento en la misma base que el pedido. Un worker (**bus-workers**) lo lee después y lo publica a Event Hubs. Así no se pierde el evento si el bus estaba caído en el momento del commit.

¿Inventario es lo mismo que el WMS?

Qué decir: No. Inventario = nuestro dominio de reservas. WMS = sistema de almacén externo (mock en el MVP).

¿Quién publica el evento canónico: el OMS o bus-workers?

Qué decir: El OMS deja el pending en outbox SQL. Quien publica a Event Hubs es **bus-workers**.

¿Service Bus ya está 100 % usado en la demo?

Qué decir: El recurso existe. Gran parte del cableado de consumidores es objetivo. En E5 usamos la dead-letter de Service Bus; el resubmit se hace en el Portal de Azure.

¿El BFF qué es?

Qué decir: Una capa del MVP que facilita la demo (portal/ops) frente a los servicios. No reemplaza APIM como gateway de borde.

¿Cuánto tiempo a producción real?

Qué decir: El MVP valida arquitectura. Producción implica WMS real, identidad completa, HA, observabilidad y las iniciativas del TO BE (meses/años), no “apagar el mock y listo”.

¿Cómo controlan costos (FinOps)?

Qué decir: SKUs chicos (APIM Developer, 1 TU Event Hubs, AKS 2 nodos), tags por ambiente, plan antes de apply, y presupuesto mensual documentado (~449 USD nube).

¿Qué pasa en un pico tipo Cyber Days?

Qué decir: Colas (Service Bus / SQS), backpressure y DLQ + replay. No prometemos absorber 240k con el SKU de demo; el patrón está listo para crecer el throughput.

¿Por qué SQL y no Cosmos / Postgres?

Qué decir: OMS/orquestador ya viven en mundo Azure SQL; outbox relacional encaja. No cambiamos de motor solo por moda en el MVP.

12. Guion 5 minutos (recordatorio)

1. Dolores: duplicados, Cyber Days, offline, tracking.
2. Multinube: ya están en 3 nubes → unimos con eventos.
3. Demo: orden → reserva → outbox → Event Hubs (+ DLQ si preguntan).
4. Diferencial: resiliencia del bus, offline móvil, CQRS.
5. Cierre: IaC + CI con plan, ~449 USD/mes, mocks = contratos legado.